SRS

Smart Cold Storage Monitor

Team Members:

Leyang Pan 003989438, Bo Sun 002032671, Zi Ye 003916934

Problem Statement:

Temperature fluctuations in cold storage environments can affect the quality and shelf life of perishable food products. Conventional temperature monitoring systems generally rely on fixed thresholds and alerts, which only identify problems after temperatures have exceeded acceptable limits.

These systems also provide limited information about how historical temperature exposure affects stored products. Consequently, operators may struggle to predict temperature-related problems, identify potentially deteriorating products, and estimate their remaining shelf life.

We propose an AI-powered cold storage monitoring system that analyzes historical temperature datasets, predicts future temperature conditions, and estimates food spoilage risk and remaining shelf life.

The project will use publicly available Kaggle datasets rather than physical sensors, allowing development and evaluation of the predictive models without hardware dependencies.

Objective / Goal:

We are developing a software-based cold storage monitor and prediction platform with two primary Machine Learning modules:

Module 1: AI Temperature Forecasting

Analyze historical cold storage temperature records and forecast future temperature changes. The module will identify potential temperature spikes before they occur and provide early warnings when forecast temperatures are expected to exceed an user set threshold.

Module 2: AI Spoilage and Shelf-Life Prediction

Analyze temperature exposure history and food product characteristics to estimate product deterioration, potential spoilage risk, and remaining shelf life.

These two modules will be integrated into a Java-based application, so that users can upload datasets, generate predictions, and review the potential impact of changes in storage conditions on food products.

We will also practice additional objectives include:

- Apply machine learning to practical cold storage management problems.
- Compare predictive models against baseline methods.
- Maintain basic prediction and activity logs aligned with relevant NIST SP 800-53 auditing principles.

Primary Use Case:

User: Cold storage manager

Use Cases:

1. The user uploads a CSV dataset containing historical cold storage temperature records.
2. The application validates and processes the dataset, identifying timestamps, temperature readings, and available environmental variables.
3. The temperature forecasting module analyzes recent temperature trends and predicts temperature conditions over the next configurable period.
4. The application displays historical and predicted temperatures, explaining potential threshold violations.
5. The user selects a food product, enters its relevant storage information, and associates it with a temperature history.
6. The spoilage and shelf-life prediction module evaluates the product's historical temperature exposure and estimates its deterioration level or remaining shelf life, using an appropriate trained model.
7. The application presents the predicted shelf-life estimate, supporting factors, and uncertainty or model limitations.
8. The application stores prediction results and relevant user actions for future review.

Scope of the Project:

Module 1 — AI Temperature Forecasting

- Historical temperature dataset upload and processing
- Temperature data cleaning and missing-value handling
- Time-series feature engineering
- Machine learning model training and evaluation
- Short-term temperature forecasting
- Configurable prediction horizons
- Detection of predicted temperature threshold violations
- Comparison of forecast performance against baseline methods

Module 2 — AI Spoilage and Shelf-Life Prediction

- Food product profile management
- Association of products with historical temperature records
- Analysis of cumulative temperature exposure
- Food deterioration or spoilage-risk estimation

- Remaining shelf-life prediction where suitable training labels or validated models are available
- Comparison of different storage temperature scenarios
- Presentation of prediction confidence, uncertainty, and limitations

Out of scope

- Physical sensors, IoT devices, and live data collection
- Control of refrigeration equipment
- Comparison of hypothetical storage scenarios ("what if the cooler had stayed at 2°C")
- Pharmaceutical products, and food safety or regulatory certification
- User accounts and role-based access (the application has a single local user)
- Cloud deployment and automatic model retraining

Tech Stack:

Frontend:

- Language: Java
- Framework: JavaFX
- UI Components: JavaFX controls and FXML
- Data Presentation: JavaFX TableView, Label, TextField, ComboBox, and Alert dialogs
- Styling: JavaFX CSS (basic styling only)

Backend:

- Language: Java
- Framework: Spring Boot
- API: RESTful API
- Data Processing: CSV parsing, validation, and data transformation
- ML Integration: HTTP communication with Python prediction endpoints

Machine Learning:

- Language: Python
- Libraries: pandas, NumPy, scikit-learn, XGBoost
- Time-Series Modeling: Feature-based regression and time-series forecasting methods
- Spoilage Modeling: Regression and classification models
- Model Evaluation: MAE, RMSE, and task-specific classification metrics
- Model Interface: FastAPI

Database:

- Primary Database: MongoDB for lightweight data storage
- Stored Information: Dataset metadata, product profiles, prediction history, and audit records

External Data Sources:

- Primary Dataset Source: Kaggle
- Temperature Data: Publicly available refrigeration, cold storage, or comparable environmental temperature time-series datasets
- Spoilage Data: Publicly available food freshness, spoilage, or shelf-life datasets with relevant labels

Development Tools:

- Version Control: Git and GitHub
- Java IDE: IntelliJ IDEA
- Python IDE: VS Code
- Java Build Tool: Maven
- Testing: JUnit 5, pytest

Hosting / Deployment

- Primary Deployment: Local execution
- Frontend: JavaFX desktop application
- Java Backend: Locally hosted Spring Boot service
- Python ML Service: Locally hosted FastAPI service
- Database: Local MongoDB instance

External Interface Requirements:

User Interfaces:

1. Main Menu
    - Provide navigation to the Temperature Forecasting and Spoilage Prediction modules.
    - Allow users to access previously saved predictions and uploaded datasets.

2. Dataset Upload Interface
    - Allow users to upload CSV files containing historical temperature records.
    - Display basic dataset information, such as filename, number of records, and available attributes.
    - Notify users of missing or invalid data.

3. Temperature Forecasting Interface
    - Allow users to select an uploaded temperature dataset.
    - Allow users to configure the forecast period and acceptable temperature threshold.
    - Display predicted temperature values in a simple table.
    - Display warning messages when predicted temperatures exceed the configured threshold.
    - Provide a summary of predicted minimum and maximum temperatures.

4. Spoilage and Shelf-Life Prediction Interface

- Allow users to select a food product or supported food category.
- Allow users to enter relevant product information and select associated temperature records.
- Display the predicted remaining shelf life or estimated spoilage risk.
- Provide a brief explanation of factors affecting the prediction.

5. Prediction History Interface
- Display previously generated predictions in a table.
- Allow users to review prediction results and timestamps.

Hardware Interfaces:
- No external hardware or physical sensors required.

Software Interfaces:
JavaFX Frontend to Spring Boot Backend
- Communicate through RESTful APIs.
- Exchange data using JSON.
- Upload CSV files using HTTP requests.

Spring Boot Backend to Python ML Service
- Communicate through HTTP REST APIs.
- Send processed temperature and product information to the appropriate ML model.
- Receive prediction results in JSON format.

Database Interface
- MongoDB stores dataset metadata, product profiles, prediction history, and audit records.
- Uploaded CSV files can be stored locally, with their file paths recorded in MongoDB.

Functional Requirements:
General System Requirements:
- Allow users to upload, validate, and select historical temperature datasets in CSV format.
- Store and display previous prediction results.
- Automatically generate audit logs for dataset operations, AI predictions, threshold warnings, and system errors.
- Record timestamps, event types, sources, associated identifiers, and operation outcomes.
- Store audit logs separately in MongoDB and prevent modification through normal application operations.
- Allow users to review audit logs in a simple table.

Module 1 — AI Temperature Forecasting:

- Use machine learning to forecast future temperatures based on historical data.
- Allow users to configure prediction periods and temperature thresholds.
- Display predicted temperatures and warnings for potential threshold violations.
- Evaluate forecasting accuracy against baseline methods.

Module 2 — AI Spoilage and Shelf-Life Prediction:

- Allow users to select food products and associate them with historical temperature records.
- Use machine learning to estimate food spoilage risk and remaining shelf life.
- Display prediction results and relevant factors affecting food deterioration.
- Evaluate model performance using appropriate metrics.

Non-Functional Requirements:

Performance: Generate predictions within 5 seconds under normal operating conditions.
Accuracy: Evaluate ML models using appropriate metrics and compare them against baseline methods.
Usability: Provide a simple JavaFX interface with text-based results and tables.
Reliability: Handle invalid datasets and prediction errors gracefully.
Security and Auditability: Maintain automatically generated, modification-restricted audit logs for important system activities, aligned with NIST SP 800-53 controls AU-2, AU-3, AU-9, and AU-12.
Maintainability: Keep Java and Python components modular and support local deployment without cloud dependencies.

Estimated Timeline:

1. Planning and Design (2 Weeks)

Week 1:

- Finalize project requirements and SRS documentation.
- Research and select suitable Kaggle datasets for both ML modules.
- Verify the availability of spoilage or shelf-life target labels.
- Define machine learning input features and prediction outputs.

Week 2:

- Design the JavaFX user interface.
- Design database schema and REST API structure.
- Set up GitHub repository and development environments.

2. Core Application Development (2 Weeks)

Week 3:

- Implement JavaFX frontend with basic navigation and input forms.
- Set up Spring Boot backend.
- Set up MongoDB database.
- Implement CSV upload and dataset validation.
- Implement dataset selection and management.

Week 4:

- Set up Python FastAPI service.
- Establish Java-to-Python API communication.

3. Machine Learning Development (3 Weeks)

Week 5-6:

- Clean and preprocess selected Kaggle datasets.
- Develop and evaluate the temperature forecasting model.
- Implement configurable prediction periods and temperature thresholds.
- Develop and evaluate the spoilage and shelf-life prediction model.
- Compare both models against suitable baseline methods.
- Save trained models for inference.

Week 7:

- Integrate both ML models with the Spring Boot backend.
- Display prediction results in JavaFX.
- Implement temperature warning messages.
- Implement food spoilage and shelf-life prediction forms.
- Implement prediction history and basic audit logging.

4. Testing and Quality Assurance (2 Weeks)

Week 8:

- Perform Java unit testing with JUnit.
- Perform Python model testing with pytest.
- Test REST API communication.
- Test dataset validation and error handling.
- Evaluate machine learning performance using held-out test data.

Week 9:

- Perform end-to-end system testing.
- Test both modules using sample Kaggle datasets.
- Fix integration and prediction errors.
- Improve application performance and usability.
- Prepare demonstration examples.

5. Deployment and Documentation (1 Week)

Week 10:

- Finalize local deployment of JavaFX, Spring Boot, FastAPI, and MongoDB.
- Prepare application setup and installation instructions.
- Document model training procedures and evaluation results.
- Complete technical documentation and user manual.
- Prepare project demonstration and presentation.
- Submit final project.